JS

Курс из 3 частей · для тех, кто уже знает C# и Python

Удаление, память и финал

Часть 3. Собираем полный конкурсный каталог

ЧАСТЬ 1 · ПРОЙДЕНО

Массив → createCard → render

ЧАСТЬ 2 · ПРОЙДЕНО

data.json, state, update, события

ЧАСТЬ 3 · СЕЙЧАС

Удалить, запомнить, сбросить, сдать

ТЕОРИЯ · 1

Удаление = filter по id + update()

В createCard: `<button class="delete" data-id="${p.id}">Удалить</button>`

КОД · script.js

```
function deleteProduct(id) {
  // оставляем всех, КРОМЕ товара с этим id
  allProducts = allProducts.filter((p) ⇒ p.id ≠ id);
  update(); // перерисовали с текущими фильтрами
}

deleteProduct(2); // удалим Мышь (id: 2)
console.log(allProducts.length);
```

ЧТО ПРОИЗОЙДЁТ

ДО · 6 карточек

Наушники · **Мышь** · Клавиатура · Колонка · Геймпад · Руль

ПОСЛЕ · 5 карточек · в консоли: 5

Наушники · Клавиатура · Колонка · Геймпад · Руль

⚠ Ловушка JS: индексы после удаления

Не удаляй по номеру карточки: после поиска 2-я на экране ≠ allProducts[1], а splice сдвигает все номера. id — постоянный «паспорт».

Как это в C# / Python: list.RemoveAll(p => p.Id == id) или [p for p in products if p["id"] != id].

ТЕОРИЯ · 2

Какую карточку нажали? Делегирование

КОД · script.js

```
// ОДИН обработчик на весь каталог
catalog.addEventListener("click", (event) ⇒ {
  // по чему кликнули? ищем ближайшую .delete
  const btn = event.target.closest(".delete");
  if (!btn) return; // клик мимо кнопки — выходим

  const id = Number(btn.dataset.id); // "2" → 2
  deleteProduct(id);
});
```

ЧТО ПРОИЗОЙДЁТ

#catalog — ловит клик ✋

.card · Мышь · 1490 ₽

Удалить (data-id="2") ← клик

Клик «всплывает» от кнопки вверх до #catalog

event.target — то, по чему кликнули. **closest(".delete")** — «я сам или ближайший родитель с этим классом».

Number(...): dataset всегда строка, а id — число. "2" === 2 → **false**, и ничего не удалится!

Как это в C# / Python: один обработчик на много кнопок и проверка sender в C#, bind на родительском фрейме в tkinter.

ТЕОРИЯ · 3

localStorage — записная книжка браузера

КОД · можно прямо в консоли F12

```
// хранит ТОЛЬКО строки, по ключу
localStorage.setItem("shop", "Гаджет-шоп");
console.log(localStorage.getItem("shop"));
console.log(localStorage.getItem("nope"));

// массив → строка при записи, обратно — при чтении
const list = [{ id: 1, title: "Мышь" }];
localStorage.setItem("catalog", JSON.stringify(list));
const text = localStorage.getItem("catalog");
console.log(JSON.parse(text)[0].title);
```

ЧТО ПРОИЗОЙДЁТ

```
▸ Console
Гаджет-шоп
null
Мышь
```

F12 → APPLICATION → LOCAL STORAGE

Key	Value
shop	Гаджет-шоп
catalog	[{"id":1,"title":"Мышь"}]

Живёт после перезагрузки и закрытия браузера. Без stringify сохранится бесполезное **"[object Object]"**.

Как это в C# / Python: как json.dump в файл и json.load обратно, или File.WriteAllText + JsonSerializer.

ПРАКТИКА

Каталог помнит удалённое + «Сбросить»

В deleteProduct сразу после filter добавь save(); — и каждое удаление запомнится.

КОД · script.js

```
const KEY = "catalog";
function save() {
  localStorage.setItem(KEY, JSON.stringify(allProducts));
}
async function loadProducts() {
  const saved = localStorage.getItem(KEY); // или null
  if (saved) {
    allProducts = JSON.parse(saved); // берём своё
  } else { // первый заход
    const response = await fetch("data.json");
    allProducts = await response.json();
  }
  update();
}
```

КНОПКА «СБРОСИТЬ»

```
resetBtn.addEventListener("click", () ⇒ {
  localStorage.removeItem(KEY);
  loadProducts(); // снова из data.json
});
```

ЧТО ПРОИЗОЙДЁТ

1. Первый заход: пусто → fetch → 6 карточек
2. Удалил Мышь → save() → сохранено 5
3. F5 → сохранение есть → 5, Мышь не вернулась
4. «Сбросить» → removeItem → fetch → снова 6

Как это в C# / Python: при старте программы: есть файл сохранения — читаем его, нет — берём стандартные данные.

ФИНАЛЬНЫЙ ПРОЕКТ

Полный каталог — как на конкурсе

Задание: доведи свой проект до конца

1. data.json (8+ объектов) → fetch → карточки.
2. Поиск, фильтр по категориям и сортировка — вместе.
3. Кнопка «Удалить» на карточке: data-id + делегирование.
4. save() после каждого изменения, при старте — чтение из localStorage.
5. Кнопка «Сбросить» возвращает исходный data.json.
6. Счётчик «Найдено: X из Y» и «Ничего не найдено».
7. Бонус: ♥ «Избранное» — поле favorite, переключение через find, фильтр по нему.

Чек-лист самопроверки

- Удалил → F5 → товар не вернулся
- Удаление во время поиска убирает именно ту карточку
- «Сбросить» возвращает все товары
- В Application → Local Storage видна запись
- Поиск, фильтр, сортировка и удаление не мешают друг другу
- В консоли нет красных ошибок
- Каждая функция делает одно дело
- Можешь объяснить вслух каждую строку

Решение для сравнения: папка lesson3-solution — «Книжная полка» целиком, с бонусным «Избранным».

⚠ ШПАРГАЛКА ДЛЯ КОНКУРСА

Ошибки, команда и отладка

Текст в консоли → что это значит

`Cannot read properties of null`
→ querySelector не нашёл элемент: опечатка в id или нет defer

`products is not defined`
→ опечатка в имени или переменная живёт в другой функции

`Assignment to constant variable`
→ значение меняется — нужен let, а не const

`Unexpected token … in JSON`
→ сломан data.json: лишняя запятая или кавычки ''

`blocked by CORS policy`
→ открыли двойным кликом: нужен Live Server

`data.filter is not a function`
→ в data не массив: забыли await или не тот JSON

Команда: договоритесь заранее

- id и классы: #catalog, #search, .card, .delete, data-id
- Формат data.json: поля, типы (id и price — числа), пример
- Кто какой файл правит: HTML/CSS — они, script.js — ты

F12: где что искать

- **Console** — ошибки и твои console.log
- **Elements** — появились ли карточки и их классы
- **Network** — загрузился ли data.json (200 или 404)
- **Application** — что лежит в localStorage

Главное правило: читай красную строку целиком и кликай по ссылке script.js:42 справа — она ведёт прямо к ошибке.

ПРОВЕРЬ СЕБЯ

Три задачки без подсказок

1

```
const id = btn.dataset.id;
allProducts = allProducts.filter((p) ⇒ p.id ≠= id);
```

Найди ошибку

Кнопка нажимается, ошибок нет, но ничего не удаляется. Почему?

2

```
localStorage.setItem("catalog", allProducts);
console.log(localStorage.getItem("catalog"));
```

Что выведет консоль?

И что нужно добавить, чтобы сохранение работало?

3

```
// Обработчик повесили на КАЖДУЮ кнопку «Удалить»
// один раз при загрузке страницы.
// Потом пользователь ввёл текст в поиск.
```

Объясни

Что сломается после поиска и почему? Как это исправить?

Ответы (не подглядывай!): 1) dataset.id — строка "2", а p.id — число 2, поэтому !== всегда true. Нужно Number(btn.dataset.id). 2) [object Object], [object Object],... — забыли JSON.stringify. 3) render пересоздал карточки, у новых кнопок нет обработчиков. Решение — один обработчик на #catalog.

ИТОГИ ВСЕГО КУРСА

Что ты теперь умеешь

- ✓ Подключать JS и отлаживать код в F12
- ✓ Рисовать карточки из массива объектов
- ✓ Загружать data.json через fetch + async/await
- ✓ Искать, фильтровать и сортировать по кнопкам
- ✓ Удалять по id через делегирование событий
- ✓ Запоминать изменения в localStorage и сбрасывать их

ДАЛЬШЕ · КОНКУРС

План на день конкурса

1. Договориться о data.json, id и классах
2. Карточки из массива → потом fetch
3. Поиск, фильтр, сортировка
4. Удаление и localStorage
5. Чек-лист и F12 перед сдачей

Удачи на конкурсе! 🚀